Lecture I - Welcome and Introduction

Applied Optimization with Julia

Dr. Tobias Vlćek
University of Hamburg

About this Course

About me

- **Field:** Optimizing and simulating complex systems
- **Languages of choice:** Julia, Python and Rust
- **Interest:** Modelling, Simulations, Machine Learning
- **Teaching:** OR, Algorithms, and Programming
- **Contact:** tobias.vlcek@uni-hamburg.de



I really appreciate active participation and interaction!

Learning Objectives

After today, you will be able to:

- [ ] Name the components of an optimization model
- [ ] Formulate a small transport problem as a linear model
- [ ] Adapt a model to a new objective
- [ ] Install Julia and Zed on your computer

Course Structure

Lectures

- Every Tuesday between 10.15 AM and 11.45 AM
- First four lectures repeat modelling and programming
- Later lectures discuss practical problems and implementation
- Lectures are interactive → We discuss approaches!
- Communication takes place via Moodle and E-Mail

Tutorials

- Tutorials every Friday between 8.15 AM and 9.45 AM
- In these tutorials we are working on assignments
- Please bring a laptop with Windows, macOS, or Linux!
- This Friday there is **no tutorial**!

Assignments

- Based on applied problems of the lecture
- Up to 3 students can solve assignments together
- Submitted solutions earn bonus points for the exam
- Max. 0.5 point per tutorial

Bonus points only count if the exam is passed (4.0 or better)!

Course Objective

Applied Optimization

- Real-world problems can be addressed with models
- Our objective is to foster your interest in the topic
- Enable you to recognize and solve problem structures
- Includes problem understanding and implementation

Research in Operations Research

- Part of the University of Hamburg Business School
- Aiming to solve real-world problems
- Or improving our theoretical understanding
- Publication in journals and conferences

We are also happy to supervise Bachelor and Master theses!

National and international journals

- European Journal of OR
- Journal of the Operational Research Society
- Journal on Applied Analytics
- Management Science
- Operations Research
- OR Spectrum

Applications

Brewery Production Planning



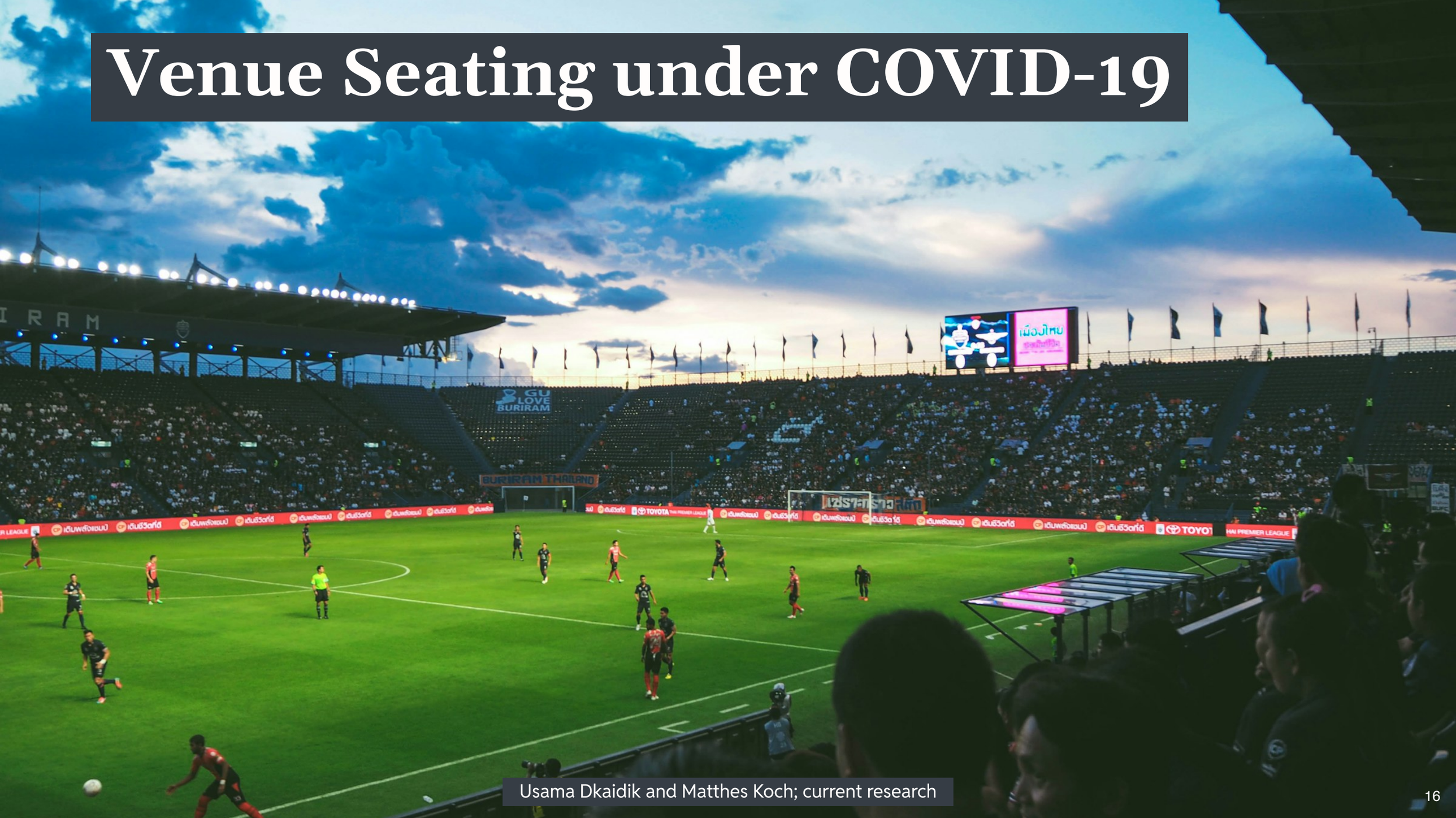
Mickein et al. (2022)

Police Service District Planning



Vlćek et al. (2024)

Venue Seating under COVID-19



Usama Dkaidik and Matthes Koch; current research

Metro Inflow Management



Vlćek et al. (2025)

Split-Order Minimization

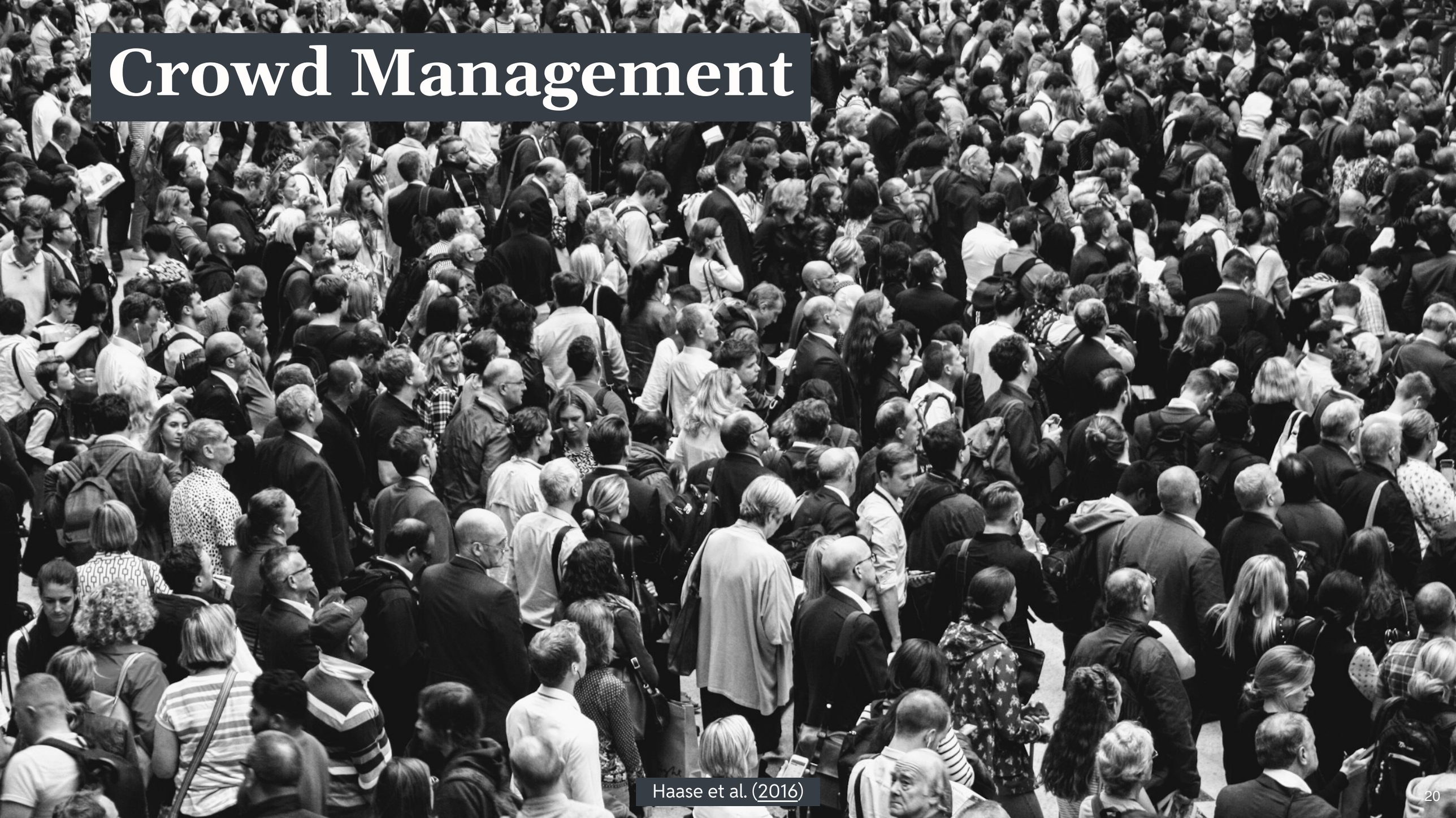


Vlćek and Voigt (2024)

Library Delivery Routing



Crowd Management



Haase et al. (2016)

Lecture Preview

Part I

1. Welcome and Introduction
2. First Steps in Julia
3. Packages and Data Management
4. Modelling with JuMP

Part II

5. Beer Production Planning
6. Minimizing Split Orders in E-Commerce
7. Periodic Library Routing
8. Police Districting

Part III

9. Safety Planning for the Islamic Pilgrimage in Mecca
10. Intermission
11. Arena Seat Planning under Distancing Rules
12. Passenger Flow Control in Urban Rail
13. Recap and Discussion

The Julia Language

Choice of Programming Language



Question: Have you ever heard of Julia?

Why Julia?

- **Designed to be:**
  - as general as Python
  - as statistics-friendly as R
  - as fast as C++!



Allows for fast data workflows, particularly in scientific computing!

Syntax

- Dynamically-typed syntax just like Python
- Similar to R, Matlab and Python - **not like C++**
- In comparison, accessible and easy to learn!
- **No need to worry about memory management!**

JuMP

- Package for **algebraic modeling** in Julia
- Simplifies solving complex optimization problems
- Provides a high-level, user-friendly interface
- Useful for operations research and data science



JuMP is an alternative to Pyomo, GAMS, and AMPL!

Must it be a new language?

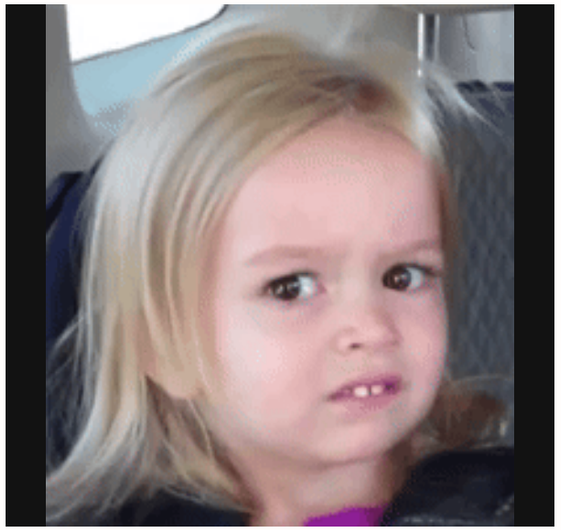
- Yes, but no need to worry!
- Julia is quite similar to Python and R
- We will learn the syntax together in the first part
- It is helpful to switch languages from time to time

Algebraic Modeling

Do you have experience with algebraic modeling?

What is algebraic modeling?

- A “mathematical language” for optimization problems
- Allows for describing complex systems and constraints
- Based on linear algebra (Equations and Inequalities)



How to learn algebraic modeling?

- **Practice, practice, and practice!**
- Understand standard models and their approach
- Develop an understanding of constraints
- Understand the structure of a model's solution space
- Use available algorithms to determine solutions

Central Questions

- What is to be decided?
- What is relevant to the decision?
- What information is given and relevant?
- What parameters (data) are needed?
- Which variables and of which type are needed?



Modeling is a creative process!

Model Components

1. Sets
2. Parameters
3. Decision variables
4. Objective function
5. Constraints



We will go through these components step by step in each lecture!

Linear Optimization Model

Basic Model Formulation

$$\text{Maximize} \quad F = \sum_{j \in \mathcal{J}} c_j \times X_j$$

subject to

$$\sum_{j \in \mathcal{J}} a_{i,j} \times X_j \leq b_i \qquad \forall i \in \mathcal{I}$$

$$X_j \geq 0 \qquad \forall j \in \mathcal{J}$$

Model Components

$$\begin{aligned}
\mathcal{I} &: \text{set of resources, indexed by } i, \\
\mathcal{J} &: \text{set of activities, indexed by } j, \\
F &: \text{Objective function variable}, \\
X_j &: \text{decision variables}, \\
c_j &: \text{objective function coefficients}, \\
a_{i,j} &: \text{parameters}, \\
b_i &: \text{parameters}
\end{aligned}$$

Question: Have you ever seen something like this before?

What is this good for?

- **Good Question!** A lot of things:
  - Modeling real-world problems
  - Solving complex systems
  - Optimizing resource allocation
  - Decision-making under constraints
  - Simulation and prediction

From Abstract to Concrete

We've just seen the general structure:

- Sets ($\mathcal{I}, \mathcal{J}$)
- Parameters ($c_j, a_{i,j}, b_i$)
- Decision variables (X_j)
- Objective function and constraints

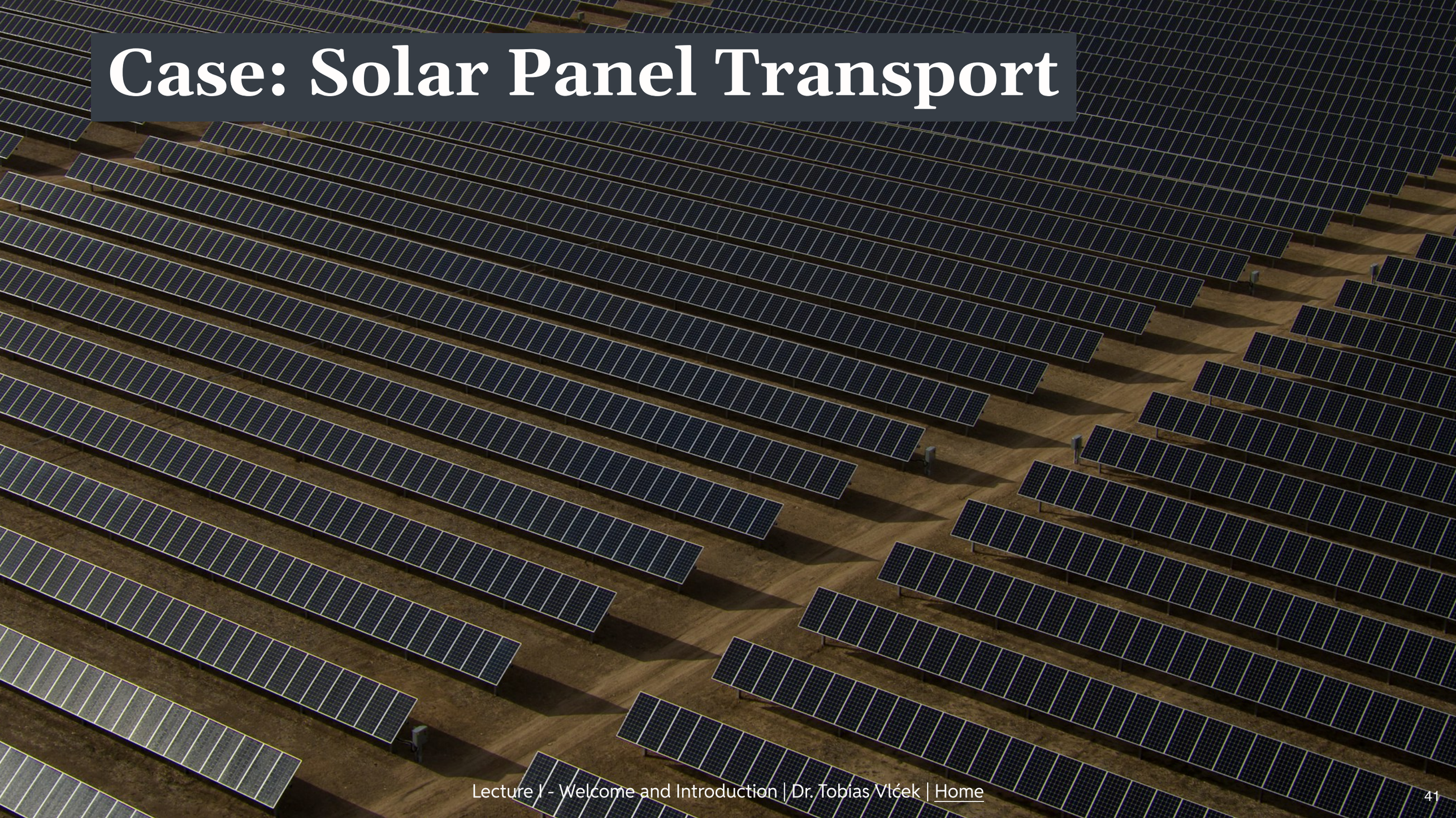
Now let's see how this works with a real problem!



Watch for these components as we build our first model together.

Solar Panel Transport

Case: Solar Panel Transport



Description

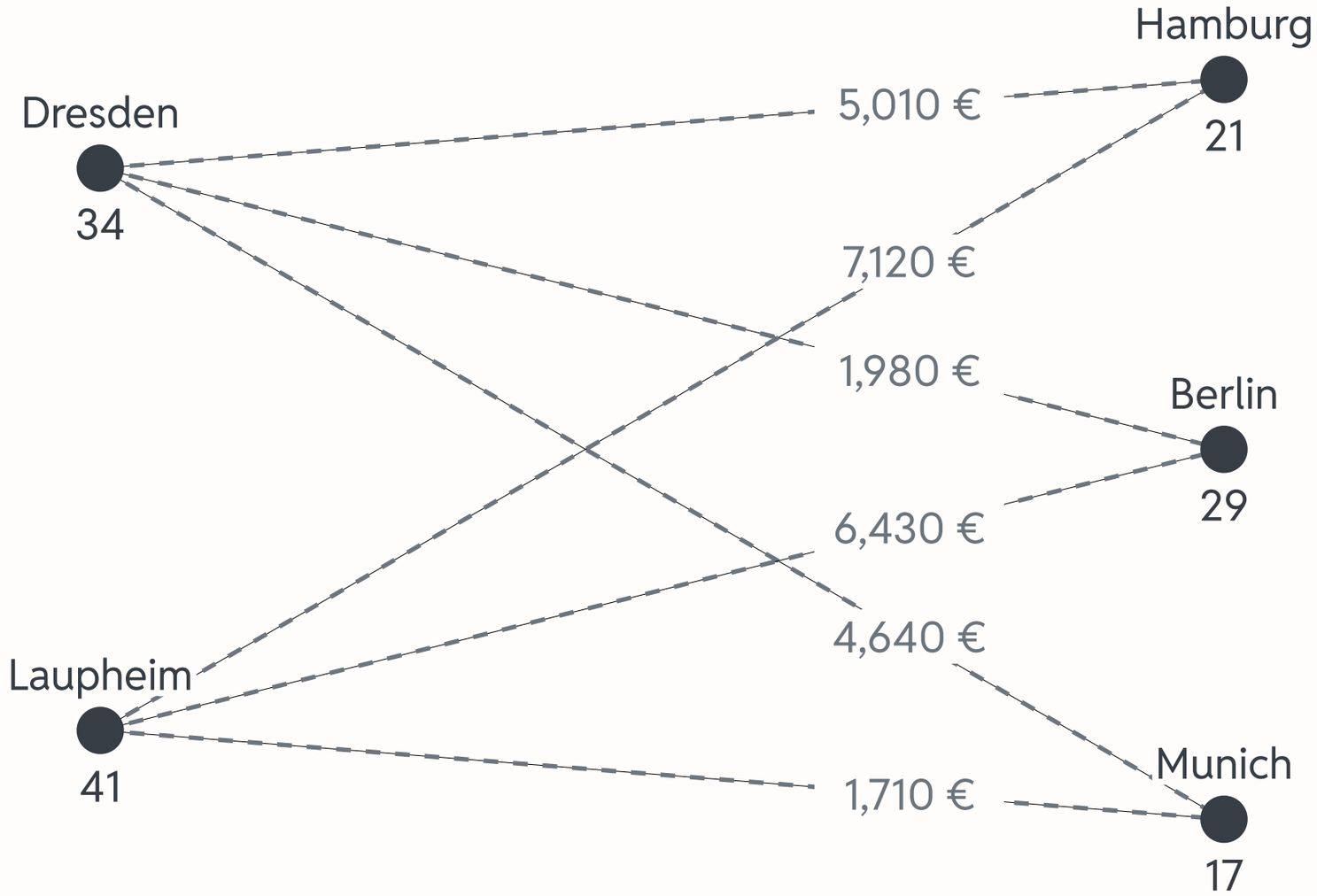
A company is producing **solar panels** in Dresden and Laupheim and has to transport them to new solar farms near Hamburg, Munich, and Berlin. The quantities offered and demanded (truckloads) and the transport costs per truckload in Euro are summarized in the following table.

Transport Costs

From / To	Hamburg	Munich	Berlin	Available
Dresden	5010	4640	1980	34
Laupheim	7120	1710	6430	41
Demand	21	17	29	

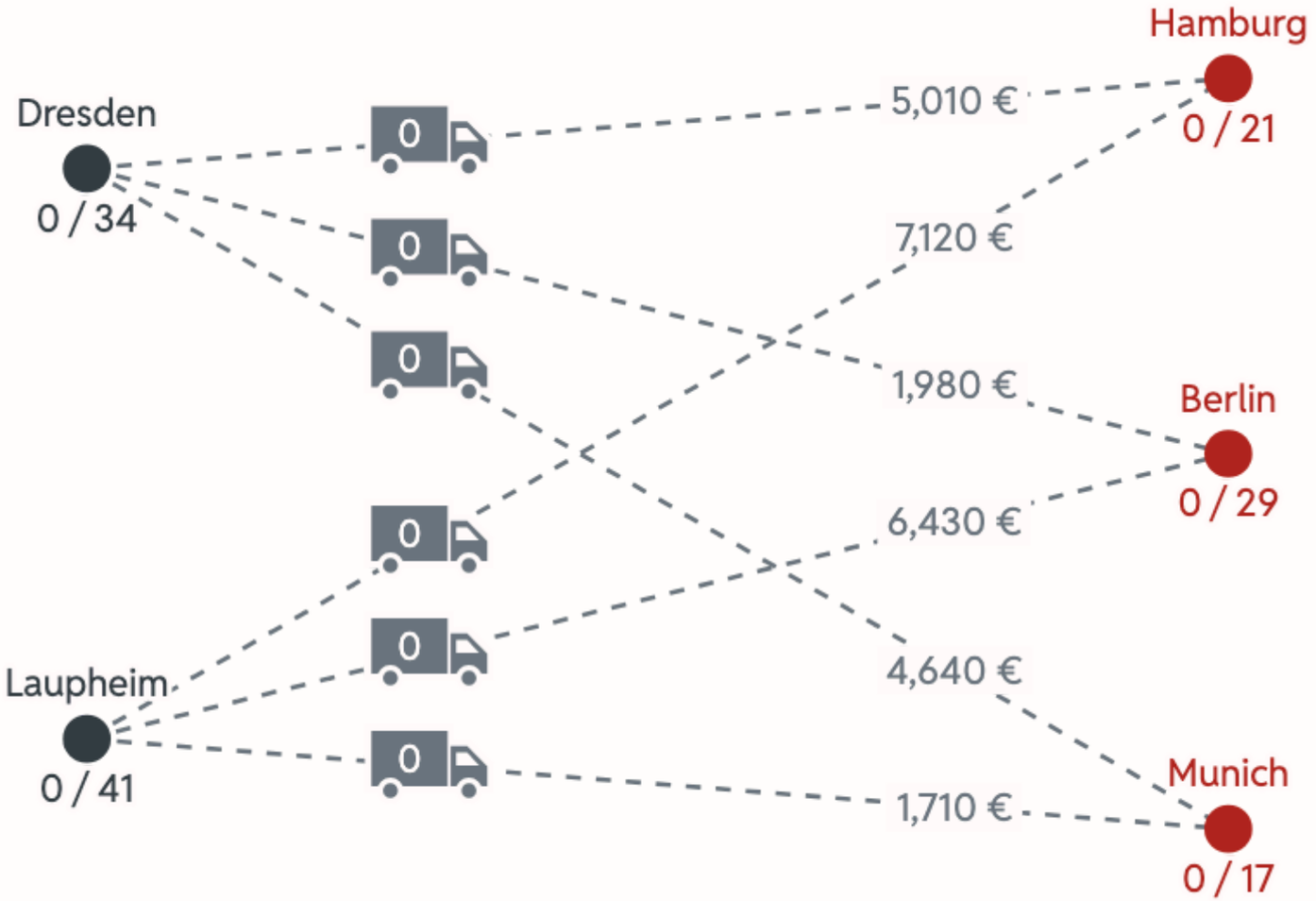
Example: A truckload from Dresden $i = 1$ to Munich $j = 2$ costs $c_{1,2} = 4640$ Euro. Moreover, it is necessary to **fulfil all customer demands**, as the contract has been signed.

Graphical Illustration



Your Turn

Meet every farm's demand at the lowest cost. Tap a truck, then use the buttons to change its load.



You 0 €

Hamburg is 21 truckloads short

Optimize

Reset

New

Problem Structure

What are we trying?

First, we always need to understand the objectives.

Question: What are our possible objectives?

Minimizing the **transport costs** over all truckloads while **meeting the demand** within the **available supply**.

Let’s break it down step by step!

Sets

Remember, sets are **collections of elements**

Question: What sets are needed?

$$\mathcal{I} : \text{Set of production sites, indexed by } i \in \{1, \ldots, |\mathcal{I}|\},$$
$$\mathcal{J} : \text{Set of customers, indexed by } j \in \{1, \ldots, |\mathcal{J}|\}.$$



We often use **plural** names for sets and a calligraphic letter, e.g., $\mathcal{I}$ and $\mathcal{J}$.

Parameters

Parameters are **fixed values that are given.**

Question: What parameters are needed?

$$
\begin{aligned}
c_{i,j} &: \text{Costs per truck load for transport from } i \text{ to } j, \\
a_i &: \text{Available truck loads at } i, \\
b_j &: \text{Customer demands at } j.
\end{aligned}
$$



We usually use the corresponding lower-case letter, e.g., c, a, and b.

Decision Variable

- Decision variables are the values we are trying to find
- Here, our objective is to minimize the transport costs

Question: What decision variables are needed?

$$X_{i,j} : \text{Trucks that deliver panels from site } i \text{ to customer } j.$$



We use upper-case letters to distinguish variables from parameters, e.g., X.

These are our building blocks!

Objective Function

- The objective function is the value we are trying to minimize (or maximize)
- Formalized as a sum of decision variables and parameters

Question: Do you remember the objective?

Minimizing the Transport Costs

Minimize the transport costs over all truckloads while meeting customer demand within the available supply from production sites.

Question: How can we write this down?

$$\text{Minimize} \quad \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} c_{i,j} \times X_{i,j}$$

Constraints

- Constraints are **conditions that must be met**
- They limit the solution space!

Question: Objective value without the supply and demand constraints?

- The value is **zero**
- Cheapest option: we simply transport nothing at all!

Question: What constraints are needed?

Supply Constraints

Ensure that the number of panels transported from a location **does not exceed** the available panels.

Question: How can we formalize this?

$$\sum_{j \in \mathcal{J}} X_{i,j} \leq a_i \quad \forall i \in \mathcal{I}$$

Demand Constraints

Ensure that the demand of each customer is covered.

Question: Any ideas?

$$\sum_{i \in \mathcal{I}} X_{i,j} = b_j \quad \forall j \in \mathcal{J}$$

Non-negativity Constraints

Ensure **no negative number of truckloads** are transported.

Question: Does anyone have an idea how to write this down?

$$X_{i,j} \geq 0 \quad \forall i \in \mathcal{I}, \forall j \in \mathcal{J}$$

Transport Problem

The complete model can then be written as:

$$\text{Minimize} \quad F = \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} c_{i,j} \times X_{i,j}$$

$$\text{subject to:} \quad \sum_{j \in \mathcal{J}} X_{i,j} \leq a_i \qquad \forall i \in \mathcal{I}$$

$$\sum_{i \in \mathcal{I}} X_{i,j} = b_j \qquad \forall j \in \mathcal{J}$$

$$X_{i,j} \geq 0 \qquad \forall i \in \mathcal{I}, \forall j \in \mathcal{J}$$

Inequality Constraints

Question: Could we replace $=$ by $\geq$ in the demand constraint?

- Yes, we could!
- We could deliver more than the demand
- But this would not happen here

Question: Why won't we transport more than the demand?

- Due to the associated costs!

The Optimal Solution

Route	Truckloads	Cost
Dresden → Hamburg	5	25,050
Dresden → Berlin	29	57,420
Laupheim → Hamburg	16	113,920
Laupheim → Munich	17	29,070

The minimal total cost is **225,460 Euro**.



In a few weeks, you will compute this solution yourself — with just a few lines of Julia code!

Any questions?

Profit Maximization

Description

Unfortunately, the margins on solar panels are low. After the previous contract has been fulfilled, the company will produce the **same number of panels** as before. In addition, all three customers want to order the **same number of truckloads** with solar panels again. The revenue per truckload of panels is 11,000 Euros. The complete production of a truckload of solar panels, including materials, costs 6,300 Euros.

New Objective

In the new contract, the company wants to **maximize its profits** while the demand does not have to be fulfilled.

Question: What changes are necessary?

- We need to change the objective function
- We need to change some parameters
- We need to adjust some constraints

Question: Does our decision variable change?

- No, we still transport truckloads of solar panels

New Parameters

$$r : \text{Revenue per truckload of solar panels,}$$
$$k : \text{Production costs per truckload of solar panels.}$$



We use k here, as $c_{i,j}$ already stands for the transport costs!

Question: What is the profit per truckload of solar panels?

$$p = r - k$$

Former Model

$$\text{Minimize} \quad F = \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} c_{i,j} \times X_{i,j}$$

$$\text{subject to:} \quad \sum_{j \in \mathcal{J}} X_{i,j} \leq a_i \qquad \forall i \in \mathcal{I}$$

$$\sum_{i \in \mathcal{I}} X_{i,j} = b_j \qquad \forall j \in \mathcal{J}$$

$$X_{i,j} \geq 0 \qquad \forall i \in \mathcal{I}, \forall j \in \mathcal{J}$$

Question: What do we need to change here?

New Model

$$\text{Maximize } F = \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} (p - c_{i,j}) \times X_{i,j}$$

$$\text{subject to: } \sum_{j \in \mathcal{J}} X_{i,j} \leq a_i \qquad \forall i \in \mathcal{I}$$

$$\sum_{i \in \mathcal{I}} X_{i,j} \leq b_j \qquad \forall j \in \mathcal{J}$$

$$X_{i,j} \geq 0 \qquad \forall i \in \mathcal{I}, \forall j \in \mathcal{J}$$

Who Receives No Panels?

Question: One customer will not receive a single panel. Who and why?

- Profit per truckload: $p = 11000 - 6300 = 4700$ Euro
- Dresden → Hamburg: $4700 - 5010 = -310$ Euro
- Laupheim → Hamburg: $4700 - 7120 = -2420$ Euro

- Serving **Hamburg** loses money on both routes!
- This is exactly why the demand constraint had to become $\leq$

The Optimal Profit Plan

Route	Truckloads	Profit
Dresden → Berlin	29	78,880
Laupheim → Munich	17	50,830
→ Hamburg	0	0

The maximal profit is **129,710 Euro**.

> ⓘ With the former = demand constraint, the model would be **forced** into the unprofitable deliveries to Hamburg!

Model Reflection

Take a moment to think about what we just built:

Size | Scaling | Assumptions | Variations | Integers

How many decision variables does this problem have?

$|\mathcal{I}| \times |\mathcal{J}| = 2 \times 3 = 6$ variables

Each represents a shipping route from a production site to a customer.

What Did We Learn?

Modeling Process:

- Define the problem clearly
- Identify decision variables
- Formulate obj. function
- Add necessary constraints
- Verify completeness

Key Insights:

- Sets organize our indices
- Parameters hold data
- Variables = decisions
- Constraints limit feasibility
- Obj. drives optimization



This systematic approach works for **any** optimization problem!

Do you have any questions?

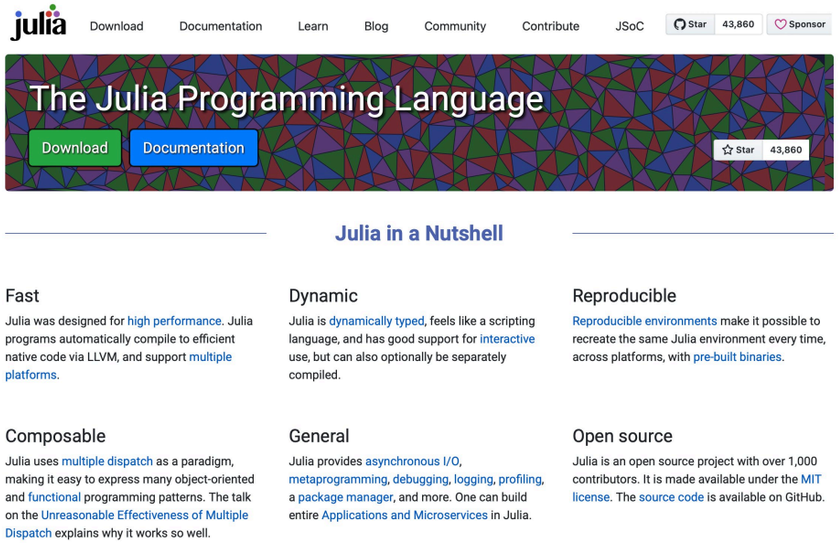
Installing Julia

Download and Install Julia



To prepare for the upcoming lectures, we start by installing the Julia Programming Language and an Integrated Development Environment (IDE) to work with Julia.

Installing Julia



- Head to julialang.org and follow the instructions.
- The easiest way to install Julia is via the shell/terminal
- Later, you can then manage Julia with `juliaup`



If you are ever asked to add something to your “PATH”, do so!

Zed

- Next, we are going to install Zed, the editor of this course
- macOS: `brew install --cask zed` (or the installer from zed.dev)
- Windows: installer from zed.dev/windows
- Linux: `curl -f https://zed.dev/install.sh | sh`



On Windows, Zed needs a DirectX-11-capable GPU. If it only shows a blank window, use VS Code with the “Julia (Julia Language Support)” extension instead: everything in this course works there too.

Verify the Installation

- Start Zed and take a look around
- Open the command palette: `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS)
- Choose **zed: extensions** and search for “Julia”
- Install the Julia extension for syntax support

> ⚠ Any problems? Ask me for help!

Create a new file

- Create a new file with a ".jl" ending
- Save it somewhere on your computer
- e.g., in a folder that you will use in the course

```
1 print("Hello World!")
```

```
Hello World!
```

- Open a terminal in that folder and run the file with:
- `julia test.jl`

Everything working?

- If the terminal prints `Hello World!` → perfect!
- If not, the terminal likely **cannot find the path** to Julia
- Close and reopen the terminal after installing Julia, and say yes to any “PATH” question during installation
- Then try to run the file again



Don’t worry if it is not running right away. We will fix this together!

Installation Checklist

Before the next lecture, try to ensure you can:

- [ ] Open Zed
- [ ] See the Julia extension installed in **zed: extensions**
- [ ] Create a new `.jl` file
- [ ] See syntax highlighting in your Julia file
- [ ] Run code and see output in the terminal

> ⚠ **Having trouble?** We will fix your issues together in the next lecture!

Starting with Julia

How to get started?

- Learning a new programming language is a daunting task
- It is best to start with some small, interactive problems
- Then, slowly increase the scope of the tasks
- We will do this **together in class**!

> **And that's it for today's lecture!**
> We now have covered a first introduction and are ready to start solving some problems in the upcoming lectures.

Questions?

Literature

Literature

For interesting literature to learn more about Julia, take a look at the literature list of this course.

References

Haase, Knut, Habib Zain Al Abideen, Salim Al-Bosta, et al. 2016. “Improving Pilgrim Safety During the Hajj: An Analytical and Operational Research Approach.” *Interfaces* 46 (1): 74–90.

Mickein, Markus, Matthes Koch, and Knut Haase. 2022. “A Decision Support System for Brewery Production Planning at Feldschlösschen.” *INFORMS Journal on Applied Analytics* 52 (2): 158–72.

Vlćek, Tobias, Usama Dkaidik, Knut Haase, et al. 2025. *Real-Time Passenger Flow Balancing for Urban Rail Transit: A Computationally Efficient Optimization Approach for Peak Demand Management*. SSRN preprint, to be submitted to IISE Transactions. https://doi.org/10.2139/ssrn.5225835.

Vlćek, Tobias, Knut Haase, Malte Fliedner, and Tobias Cors. 2024. “Police Service District Planning.” *OR Spectrum* 46 (4): 1029–61. https://doi.org/10.1007/s00291-024-00745-3.

References (2/2)

Vlćek, Tobias, and Guido Voigt. 2024. *Optimizing SKU-Warehouse Allocations to Minimize Split Parcels in E-Commerce Environments*. Working paper, to be submitted to Manufacturing & Service Operations Management.